

TRABAJO PRÁCTICO N°2

Análisis Comparativo: Ruby vs. Go

Gramática Formal y Algoritmos de Ordenamiento

Grupo 1 — Integrantes:

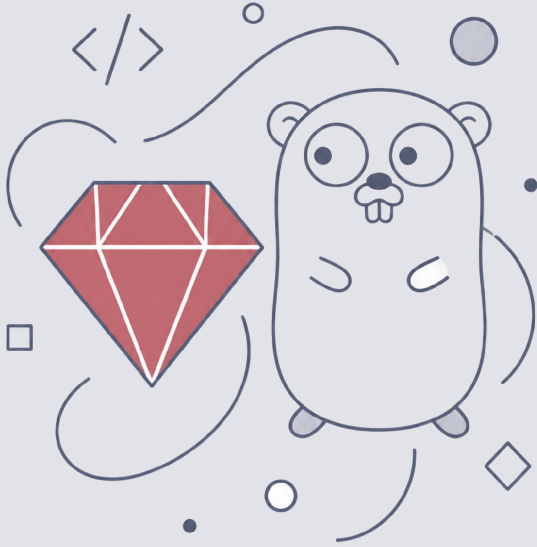
Mendoza Aguirre, Lautaro

Minaya Gutiérrez, Yair César

Raposo, Gonzalo Ezequiel

Vázquez, Nicolás

Profesora: Roxana Leituz



Ruby vs. Go

Historia, gramática formal y algoritmos de ordenamiento — Trabajo
Práctico Teórico

L1: RUBY

L2: GO

TEMA 1: ORDENAMIENTO



Introducción a Ruby

Origen

Creado por Yukihiro "Matz"
Matsumoto en Japón
(1993–1995). Ruby 0.95
lanzado en diciembre de 1995.

Motivación

Productividad, simplicidad y
felicidad del programador.
Inspirado en Perl, Smalltalk,
Eiffel, Ada, BASIC y Lisp.

Nombre

Elegido en un chat en 1993: el rubí era la piedra de nacimiento de un
colega de Matz.

Características de Ruby



Orientado a objetos Puro

Todo valor es un objeto, incluidos números y el nulo.



Tipado Dinámico

No se declaran tipos al crear variables.



Sintaxis Intuitiva

Posee una sintaxis elegante, humana y natural .



Multiparadigma

Soporta multiples paradigmas, permite multiples metodos de programacion



Garbage Collector

Gestión automática de memoria integrada.



Alta Portabilidad

GNU/Linux, UNIX, Windows, MacOS y DOS.

Características de Ruby

Variables

No hace falta declarar el tipo de las variables.



Bloques

Los bloques de código terminan con end en lugar de usar llaves.



Estructuras

Los arrays y los hashes son estructuras de datos muy utilizadas.



Impresión

puts imprime con salto de línea, print no.



Objetos

Todo es un objeto, incluyendo números y cadenas.



Métodos

Las funciones se llaman métodos y se definen con def.



Colecciones

Para recorrer colecciones se suele usar each.



Valor nil

nil representa la ausencia de valor.

Introducción a Go

Origen

Creado en Google (2007) por Robert Griesemer, Robert Pike y Ken Thompson. Código abierto en 2009; Go 1.0 en marzo de 2012.

Motivación

Resolver escalabilidad en Google: eficiencia de C++/Java con velocidad de compilación y simplicidad de Python/Ruby.

Inspiración

C (sintaxis), Pascal/Modula/Oberon (paquetes) y CSP (conurrencia). Mascota: el Gopher azul de Renée French.



Características de Go



Tipado Estático y Compilado

Verificación en compilación; genera código máquina nativo de alto rendimiento.



Concurrencia Nativa

Goroutines y Channels para aprovechar procesadores multinúcleo con mínima memoria.



POO sin Clases

Structs e interfaces; composición sobre herencia jerárquica.



Garbage Collector

Optimizado para baja latencia en tiempo real.



Sintaxis Minimalista

Solo 25 palabras clave; `gofmt` unifica el estilo automáticamente.



Compilación Rápida

Millones de líneas se compilan en segundos.

Pilares de Go

Compilado

Ejecutables rápidos y de alto rendimiento.



Sintaxis Simple

Fácil de aprender y mantener.



Concurrencia Nativa

Ejecución eficiente de múltiples tareas.



Tipado Estático

Detección temprana de errores.



Uso en Backend

Ampliamente utilizado en desarrollo backend.



Software de Alto Rendimiento

Aplicaciones rápidas, eficientes y robustas.



Gramática Formal (BNF) - Ruby

BNF de RUBY (Resumida)

```
<programa> ::= <lista_expr>
<lista_expr> ::= <expr> [ <terminador> <lista_expr> ]
<expr> ::= <asignación> | <llamada_metodo> |
<estructura_control> | <literal> |
<def_clase> | <def_metodo> | <identificador>
<def_clase> ::= "class" <Constante> [ "<" <Constante> ]
<terminador> <lista_expr> "end"
<def_metodo> ::= "def" <identificador> [ "(" <lista_params> ")" ]
<terminador> <lista_expr> "end"
<lista_params> ::= <identificador> [ "," <lista_params> ] | ε
<estructura_control> ::= "if" <expr> [ "then" ] <terminador>
<lista_expr> [ "elsif" <expr> [ "then" ]
<terminador> <lista_expr> ]* [ "else"
<terminador> <lista_expr> ] "end"
<asignacion> ::= <identificador> "=" <expr>
<terminador> ::= ";" | "\n"
```

Gramática Formal (BNF) - Go

EBNF de GO (Resumida)

`<archivo_fuente> ::= <clausula_paquete> ";" { <importacion> ";" } { <declaracion_global> ";" }`

`<clausula_paquete> ::= "package" <identificador>`

`<importacion> ::= "import" ( <string> | "(" { <string> ";" } ")" )`

`<declaracion_funcion> ::= "func" <identificador> "(" [ <lista_parametros> ] ")" [ <tipo_retorno> ] <bloque>`

`<bloque> ::= "{" { <sentencia> ";" } "}"`

`<sentencia> ::= <declaracion> | <asignacion> | <llamada_funcion> | <estructura_if> | <retorno> | ε`

`<estructura_if> ::= "if" [ <sentencia_simple> ";" ] <expresion> <bloque> [ "else" ( <estructura_if> | <bloque> ) ]`

`<terminador> ::= ";" | "\n"`

Algoritmos de Ordenamiento

Algoritmos "Puros"

QuickSort

Pivote + recursión.

Promedio $O(n \log n)$; peor caso

$O(n^2)$. No estable.

MergeSort

Divide y fusiona. Garantiza $O(n \log n)$.

Estable, pero consume $O(n)$ extra.

Insertion Sort

Óptimo para arreglos pequeños
(<20 elem.).

Sin sobrecarga recursiva.

Implementaciones Híbridas

Go: pdqsort (v1.19+)

QuickSort + HeapSort (detecta peor caso) + Insertion Sort (≤ 24 elem.). In-place, sin estabilidad por defecto.

Ruby: IntroSort / TimSort

MRI usa IntroSort (QuickSort + HeapSort + Insertion Sort). JRuby usa TimSort (MergeSort + Insertion Sort), estable.

El Enfoque de Ruby (Array#sort)



La ESTABILIDAD es su prioridad

Algoritmo de Ordenamiento

Utiliza enfoques híbridos altamente optimizados como **IntroSort** o **TimSort** (implementado en entornos como JRuby) para garantizar la máxima eficiencia.

Objetivo Principal: **La Estabilidad**

¿Por qué?: Garantiza estrictamente que dos elementos con el mismo valor mantengan de forma exacta su orden relativo original tras el ordenamiento.

Uso Ideal: Resulta ideal al ordenar objetos complejos o registros provenientes de bases de datos donde conservar el orden parcial previo es crítico.

01-OrdenamientosRubyGo > ordenamientoRuby.rb

You, yesterday | 1 author (You)

```
1 # 1. Ordenamiento básico de un arreglo de primitivos      You, yesterday * ordenamientos _
2 numeros = [9, 2, 5, 1, 7, 5, 15, 9 , 20, 21, 1, 2, 3, 0]
3
4 numeros_ordenados = numeros.sort
5 puts "Números ordenados: #{numeros_ordenados}"
6
7 # 2. Ordenamiento de estructuras complejas (Hashes)
8 estudiantes = [
9   { nombre: "Carlos", nota: 9 },
10  { nombre: "Ana", nota: 7 },
11  { nombre: "Belen", nota: 6 },
12  { nombre: "Francisco", nota: 10 },
13  { nombre: "Paula", nota: 7 },
14  { nombre: "Romina", nota: 7 },
15  { nombre: "Jose", nota: 5 },
16  { nombre: "Nicolas", nota: 10 },
17 ]
18
19 # Al ser un algoritmo estable, Las mantendrán su orden original entre ellas.
20 estudiantes_por_nota = estudiantes.sort_by { |estudiante| estudiante[:nota] }
21
22 puts "Estudiantes ordenados por nota:"
23 pp estudiantes_por_nota
24
25
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS GIT LENS

Filter (e.g. text, lexcludeText, t...

Code

[Running] ruby "c:\Users\WoxiePC\Desktop\SintaxisDeParadigmas\01-OrdenamientosRubyGo\ordenamientoRuby.rb"

Números ordenados: [0, 1, 1, 2, 2, 3, 5, 5, 7, 9, 9, 15, 20, 21]

Estudiantes ordenados por nota:

```
{nombre: "Jose", nota: 5},
{nombre: "Belen", nota: 6},
{nombre: "Ana", nota: 7},
{nombre: "Paula", nota: 7},
{nombre: "Romina", nota: 7},
{nombre: "Carlos", nota: 9},
{nombre: "Francisco", nota: 10},
{nombre: "Nicolas", nota: 10}]
```

[Done] exited with code=0 in 0.194 seconds

El Enfoque de Go (sort.Slice)



La velocidad bruta y el manejo de memoria es su prioridad

Algoritmo de Ordenamiento

Utiliza **pdqsort** (Pattern-Defeating Quicksort), una técnica de última generación altamente optimizada para un rendimiento implacable.

Estrategia Híbrida y Objetivos

- **Motor Principal:** QuickSort para asegurar la máxima velocidad práctica.
- **Defensa de Patrones:** Cambia a HeapSort si detecta ineficiencia matemática o degradación del peor caso.
- **Arreglos Chicos:** Usa Insertion Sort para terminar rápido en colecciones pequeñas.
- **Objetivo Clave:** Máxima velocidad matemática y reordenamiento **in-place** sin usar memoria extra.

```
3 import (
4     "fmt"
5     "sort"
6 )
7
8 func main() {
9     // 1. Ordenamiento básico de un Slice (arreglo dinámico)
10    numeros := []int{9, 2, 5, 1, 7, 5, 15, 9, 20, 21, 1, 2, 3, 0}
11
12    // sort.Ints modifica el slice original directamente en memoria (in-place)
13    sort.Ints(numeros)
14    fmt.Printf("Números ordenados: %v\n", numeros)
15
16    // 2. Ordenamiento de estructuras complejas (Structs)
17    type Estudiante struct {
18        Nombre string
19        Nota   int
20    }
21
22    estudiantes := []Estudiante{
23        { "Ana", 7 },
24        { "Belen", 6 },
25        { "Francisco", 10 },
26        { "Paula", 7 },
27        { "Romina", 7 },
28        { "Jose", 5 },
29        { "Nicolas", 10 },
30        { "Carlos", 9 },
31    }
32    sort.Slice(estudiantes, func(i, j int) bool {
33        return estudiantes[i].Nota < estudiantes[j].Nota
34    })
35
36    fmt.Printf("Estudiantes ordenados por nota: %v\n", estudiantes)
37    // sort.Slice recibe el slice y una función anónima que define cómo comparar dos elementos.
38    // Nota: sort.Slice usa pqsort, por lo que NO garantiza estabilidad.
39    // Si se necesitara estabilidad estricta (que Ana siempre quede antes que Belén),
40    // Go requiere llamar explícitamente a otra función: sort.SliceStable()
41 }
```

```
[Running] go run "c:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\01-OrdenamientosRubyGo\ordenamientoGo.go"
Números ordenados: [0 1 1 2 2 3 5 5 7 9 9 15 20 21]
Estudiantes ordenados por nota: [{Jose 5} {Belen 6} {Ana 7} {Paula 7} {Romina 7} {Carlos 9} {Francisco 10} {Nicolas 10}]
```

```
[Done] exited with code=0 in 0.3 seconds
```

Resumen Comparativo (Ordenamiento)

Característica	Ruby	Go
Algoritmo Base	IntroSort / TimSort	pdqsort (Pattern-Defeating Quicksort)
Filosofía	Prioriza la estabilidad y el correcto manejo de objetos complejos.	Busca el rendimiento bruto y la máxima eficiencia.
Memoria (Espacio)	Genera un arreglo nuevo (mayor consumo de RAM).	Muta directamente in-place sin usar memoria extra.
Velocidad de Ejecución	Lenguaje Interpretado (más lento).	Compilado a código máquina (más rápido).

01-OrdenamientosRubyGo > ordenamientoMuchosNumeros.rb

```

You, yesterday | 1 author (You)
1 # 1. Generamos un arreglo con 1 millón de números aleatorios (del 1 al 1000)
2 puts "Generando 1 millón de números en Ruby..."
3 numeros = Array.new(1_000_000) { rand(1..1000) }
4
5 puts "Iniciando ordenamiento..."
6
7 # 2. Capturamos el tiempo de inicio
8 tiempo_inicio = Time.now
9
10 # 3. Ejecutamos el algoritmo interno (TimSort / IntroSort)
11 numeros_ordenados = numeros.sort
12
13 # 4. Capturamos el tiempo final
14 tiempo_fin = Time.now
15
16 # 5. Calculamos la diferencia
17
18 tiempo_total = tiempo_fin - tiempo_inicio
19 puts "Ordenamiento finalizado."
20 puts "-----"
21 puts "Tiempo de ejecución en Ruby: #{tiempo_total} segundos"
22 puts "-----"

```

You, yesterday

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS GITLENS

```

[Running] ruby "c:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\01-OrdenamientosRubyGo\
Generando 1 millón de números en Ruby...
Iniciando ordenamiento...
Ordenamiento finalizado.

```

```

Tiempo de ejecución en Ruby: 0.0709495 segundos
-----

```

```

[Done] exited with code=0 in 0.354 seconds

```

01-OrdenamientosRubyGo > OrdenamientoMuchosNumerosGoEnOTraCarpetaPorqueSinoRompe > ordenamiento

```

11 func main() {
12     fmt.Println("Generando 1 millón de números en Go...")
13
14     // 1. Generamos un Slice con 1 millón de números aleatorios
15     numeros := make([]int, 1000000)
16     for i := 0; i < 1000000; i++ {
17         numeros[i] = rand.Intn(1000) // Números del 0 al 999
18     }
19
20     fmt.Println("Iniciando ordenamiento...")
21
22     // 2. Capturamos el tiempo de inicio
23     tiempoInicio := time.Now()
24
25     // 3. Ejecutamos el algoritmo interno (pdqsort)
26     sort.Ints(numeros)
27
28     // 4. Calculamos el tiempo transcurrido
29     tiempoTotal := time.Since(tiempoInicio)
30
31     fmt.Println("Ordenamiento finalizado.")
32     fmt.Println("-----")
33     // Usamos %v para imprimir el tiempo formateado automáticamente (ej. 15.4ms)
34     fmt.Printf("Tiempo de ejecución en Go: %v\n", tiempoTotal)
35     fmt.Println("-----")
36 }
37

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS GITLENS

```

[Running] go run "c:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\01-OrdenamientosRubyGo\O
Generando 1 millón de números en Go...
Iniciando ordenamiento...
Ordenamiento finalizado.

```

```

Tiempo de ejecución en Go: 30.915ms
-----

```

```

[Done] exited with code=0 in 0.875 seconds

```

Fuentes y Preguntas

Fuentes

Ruby: [ruby-lang.org](https://www.ruby-lang.org) · BNF: docs.huihoo.com · Wikibooks · Go Spec: go.dev/ref/spec

Preguntas para el grupo

- ¿Por qué Ruby prioriza estabilidad y Go velocidad?
- ¿Cuándo elegirías `sort.SliceStable` en Go?
- ¿Cómo afecta el tipado dinámico vs. estático al ordenamiento?
- ¿Qué ventaja tiene pdqsort sobre QuickSort puro?

